

Computational Physics and Modelling – Assessment 3

Smooth Particle Hydrodynamics

Elijah Gill

27 March 2026

1 Theory

This project focuses on using Smooth Particle Hydrodynamics (SPH) to model Sod's Shock Tube, which is a simulation where a tube is split into two halves. The first having a large density ($\rho_H = 1$) and pressure ($P_H = 1$), and the second having a low density ($\rho_L = 0.25$) and pressure ($P_L = 0.1795$), separated by a discontinuity. Pressure, density and specific internal energy were related using the following equation of state [1],

$$P = (\gamma - 1)\rho u, \quad (1)$$

where P is the pressure, ρ is the density, u is the specific internal energy and $\gamma = 1.4$, is the adiabatic index for a diatomic gas.

Density is calculated using a Cubic-Spline Kernel [2],

$$W(\Delta x, h) = \frac{2}{3h} \begin{cases} 1 - \frac{3}{2}q^2 + \frac{3}{4}q^3 & \text{for } 0 \leq q \leq 1 \\ \frac{1}{4}(2 - q)^3 & \text{for } 1 < q \leq 2, \\ 0 & \text{for } q > 2 \end{cases} \quad (2)$$

where Δx is the distance between two particles, h is the smoothing length, and $q = |\Delta x|/h$. This kernel is computed over every possible pairing of particles in a system to determine each particle's density using the following equation [2],

$$\rho_a = \sum_b m_b W(\Delta x, h), \quad (3)$$

where ρ_a is the density of particle a , and m_b is the mass of a compared particle b . The summation does include a comparison with itself when $a = b$. It is also useful to define the gradient of the smoothing kernel with respect Δx ,

$$\nabla_{\Delta x} W(\Delta x, h) = \frac{2}{3h^2} \text{sgn}(\Delta x) \begin{cases} -3q + \frac{9}{4}q^2 & \text{for } 0 \leq q \leq 1 \\ -\frac{3}{4}(2 - q)^2 & \text{for } 1 < q \leq 2, \\ 0 & \text{for } q > 2 \end{cases} \quad (4)$$

where $\text{sgn}(\Delta x)$ is used as q is defined using the modulus of Δx . This expression is used in finding the values of acceleration and power for each particle [2],

$$\frac{dv_a}{dt} = -\sum_b m_b \left(\frac{P_a}{\rho_a^2} + \frac{P_b}{\rho_b^2} + \Pi_{ab} \right) \nabla_{\Delta x} W(\Delta x, h), \quad (5)$$

$$\frac{du_a}{dt} = \sum_b m_b \left(\frac{P_a}{\rho_a^2} + \frac{1}{2}\Pi_{ab} \right) v_{ab} \nabla_{\Delta x} W(\Delta x, h), \quad (6)$$

where $\frac{dv_a}{dt}$ and $\frac{du_a}{dt}$ are the acceleration and power of particle a respectively, P denotes the pressure, ρ denotes the density, and Π_{ab} represents the artificial viscosity. In the model, the shock particles experience due to the discontinuity causes them to change energy quickly, which results in large, unrealistic, oscillations at the head of the shock wave. Extra dissipative terms are added to these

equations in the form of artificial viscosity to spread the energy out between particles to stop discontinuities occurring, the artificial viscosity used in this model is,

$$\Pi_{ab} = \begin{cases} \frac{-\alpha \bar{c}_{ab} \mu_{ab} + \beta \mu_{ab}^2}{\bar{\rho}_{ab}} & \text{for } r_{ab} v_{ab} < 0 \\ 0 & \text{otherwise} \end{cases}, \text{ with } \mu_{ab} = \frac{\bar{h}_{ab} r_{ab} v_{ab}}{r_{ab}^2 + \epsilon \bar{h}_{ab}^2}, \quad (7)$$

where α , β and ϵ are constants, $\bar{c}_{ab}$ is the average speed of sound between particles a and b , $\bar{\rho}_{ab}$ is the average density between particles a and b , r_{ab} is the distance between particles, v_{ab} is the difference in velocity and $\bar{h}_{ab}$ is the average smoothing length between particles a and b . Artificial viscosity should only be active when two particles are moving towards each other.

The Velocity-Verlet (Leapfrog) method was used to simulate the shock tube over time. Positions, velocities and specific internal energies were all iterated with the following equations,

$$v_{n+1/2} = v_n + \frac{1}{2} \Delta t a(x_n), \quad (8a)$$

$$u_{n+1/2} = u_n + \frac{1}{2} \Delta t P(v_n), \quad (8b)$$

$$x_{n+1} = x_n + \Delta t v_{n+1/2}, \quad (8c)$$

$$v_{n+1} = v_{n+1/2} + \frac{1}{2} \Delta t a(x_{n+1}), \quad (8d)$$

$$u_{n+1} = u_{n+1/2} + \frac{1}{2} \Delta t P(v_{n+1}), \quad (8e)$$

where the subscript defines the stage of the step the value is from. v is the velocity, Δt is the timestep, a is the acceleration, u is the specific internal energy and P is the power.

2 Code Description

This project simulates a shock tube in C++, then exports the data to a text file, “DATA.txt”, which is then read and plotted in Python. All relevant parameters are tuneable by the user in the text file called, “INPUTS.txt”. This file only reads numbers, so any letters or special characters are ignored when reading the data. The user can also read the values of the parameters that have been read by the program, as they are printed to the terminal, allowing for easy debugging. The inputs file is separated into sections for a better user experience, the first section, “Conditional Booleans” allows the user to toggle the use of boundary particles, artificial viscosity, variable smoothing lengths and exporting data. The second section, “Tube Parameters” contains variables relevant to the tube and gas dimensions. “Particle Parameters” allows the user to change the initial densities and pressures in the tube, and “Verlet Parameters” contains the variables related to integrating the model through time, as well as a parameter to determine how often data is exported to the output file.

The standard values of each variable used are shown in the table below, these values were used for all simulations unless specified.

Table 1: Contains the standard values used in the Shock Tube simulation results shown in this report.

Parameter	Value	Parameter	Value
Boundary Particles	1 (True)	Variable Smoothing Lengths	0 (False)
Artificial Viscosity	1 (True)	Export Data	1 (True)
Tube Length	1.0	Alpha	1.0
Gamma	1.4	Beta	2.0
Smoothing Length	0.016	Eta	1.2
No. Particles	250	Epsilon	0.1
No. Boundary Particles	25	Pressure (Left)	1.0
Density (Left)	1.0	Pressure (Right)	0.1795
Density (Right)	0.25	Timestep	0.0005
Start Time	0.0	End Time	0.25

2.1 Particle System Class

The class named ‘ParticleSystem’ stores all the data that is relevant to the current state of the model. It stores parameters, such as particle positions, as vectors containing the relevant values for every particle in the system. This method of data handling is much faster than storing each particle with its own variables individually, as there is less data to be accessed at a time and the path to the relevant data is shorter, saving on computing time and RAM usage.

There are three methods associated with this class,

- **Initialise()**

This function sets up the shock tube. It takes in the number of particles for the system, splits them into the ratio ($\rho_H : \rho_L$) and calculates an equal spacing between each particle to accurately assign the correct densities on each half of the tube, as shown in figure 1.

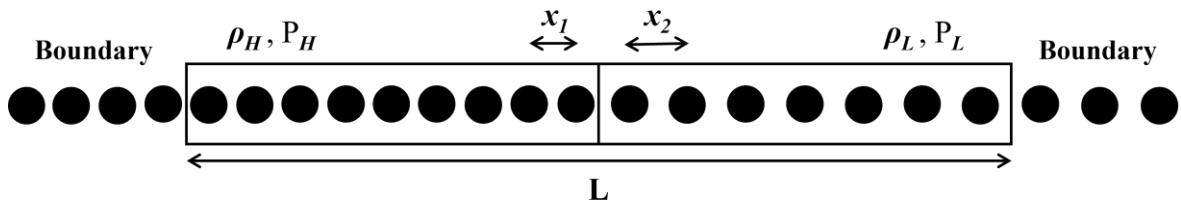


Figure 1: A diagram of the initial conditions of the shock tube, displaying equal spacing between particles on either side of the tube that is determined by the density of each half, separated by a wall at $L/2$. Boundary particles are present on the ends of the tube to avoid modelled particles from having unrealistic behaviour.

The masses of each particle are identical and are defined using the higher density multiplied by the corresponding spacing, while pressures are manually defined and specific internal energies are found by rearranging eq(1) through a dedicated function.

If boundary particles are enabled, a number of extra particles, specified by the user, on each side of the tube are defined. These particles interact with the simulation particles but never change values themselves. This stops edge particles from dropping in density as they would otherwise have no neighbours on one side.

In the code, the tube is initialised from left to right, starting with boundary particles on the left, then filling the tube itself, followed by more boundary particles on the right side of the tube.

The initialise() function also calculates the smoothing lengths if they are set to be variable, the method behind this is discussed in the Extension section of this report. If variable smoothing lengths are not enabled, which is the default for this project, a user specified value is used for every particle instead.

Finally, a total initial energy is calculated for the system using a dedicated function that sums the kinetic and specific internal energy of each particle in the system.

- **Find_params()**

This function is used to find updated values for the densities, pressures and speeds of sound for each particle in the system. Firstly, the size of the system is found and used to loop through all particles to perform a brute force search to determine how each particle interacts with each other in order to determine their new densities, which are then used to find their new pressures, and both of these used to calculate their new speeds of sound. In the same for loop, if variable smoothing lengths are enabled, new smoothing lengths are also evaluated for each particle. Doing all of these calculations over one for loop saves on computing time over doing them individually.

These values are then used to calculate each particle’s acceleration and power for use in the Verlet integration. Artificial viscosity is, by default, set to zero and only formerly calculated if it is enabled

by the user. So, if it is disabled, artificial viscosity has no impact on the acceleration and power calculations.

- **Total_energy()**

The final class method calculates the total energy of the system as the sum of kinetic and specific internal energies, this is used to check the conservation of energy, with respect to the initial total value, throughout the Verlet integration.

2.2 Smoothing Kernel Functions

- **Smoothing_kernel()**

The cubic-spline smoothing kernel is represented in the code as this function, where the distance between two particles is first normalised through division by the smoothing length to find the value of q , which is then used in the piecewise function given in eq(2).

- **Grad_smoothing_kernel()**

The gradient of the cubic-spline kernel is represented by this function. It computes eq(4) in a similar way to the `smoothing_kernel()` function; by normalising the distance between compared particles. But it also finds the sign of the displacement between the two particles before this, so that the direction forces are exerted on are correct.

2.3 Parameter Finding Functions

- **Density()**

The density function performs one iteration of the total summation described in eq(3). It is intended to be called as part of a for loop that computes every possible pair in a brute force search.

- **Eos_ideal()**

This function encodes the adiabatic ideal gas equation of state shown in eq(1), to find the pressure a particle is experiencing depending on its current density and specific internal energy.

- **Sound_speed()**

The speed of sound (c_s) in a fluid with a certain adiabatic index (γ), density (ρ) and pressure (P) is given by,

$$c_s = \sqrt{\frac{\gamma P}{\rho}}, \quad (9)$$

The function computes this value.

- **Find_u()**

Uses the adiabatic ideal gas equation of state, eq(1), rearranged to find the specific internal energy in order to find the initial energies for each particle at $t=0$.

- **Acceleration()**

The acceleration function performs one iteration of the summation in the given acceleration equation, eq(5). The negative value of the summation is baked into the function itself. The intended use is for it to be repeatedly called in a for loop, along with the `power()` function to reduce the number of for loops used, saving computing time.

- **Power()**

The power function performs one iteration of the summation in the given power equation, eq(6). The intended use is for it to be repeatedly called in a for loop, along with the `acceleration()` function to reduce the number of for loops used, saving computing time.

- **Artificial_viscosity()**

If artificial viscosity is active, this function is called to calculate its contribution to the acceleration and power for each particle, specifically computing eq(7). If artificial viscosity is not active, the default value of zero is passed to the acceleration and power functions and this function is skipped, so it has no effect on the results.

2.4 Verlet Functions

- **Verlet_step()**

A single step in the Verlet integration is performed using this function. It uses the set of eq(8) to do this, redefining the values of the acceleration and power using the `find_params( )` class method between eq(8c) and eq(8d). The total energy of the system is also calculated at the end of this step.

- **Verlet()**

The entirety of the Verlet integration is organised into this function, it is the only function that needs to be directly called to start the simulation outside of any initialisation. It calculates the size of the system as the total number of particles (including boundary particles) as this is a required parameter for many other functions. It then finds the number of iterations to run over, using the timestep, start and end times.

Values for the accelerations and power, based on initial conditions, are found before the Velocity-Verlet loop is started, which calls the `verlet_step( )` function each iteration. As well as exporting data every `n` iterations, which is specified by the user in the “`export_every`” variable. And updating a progress bar.

2.5 Miscellaneous Functions

There are some functions that do not fit neatly in any of the other categories, and so are defined in this section.

- **Read_inputs()**

This function is responsible for reading in the data given in the “INPUTS.txt” file and assigning each value to a specific variable. It contains checks to see if the input file is present and the `file.ignore( )` function is used to read in data, which delimits the inputs based on colons or a maximum character count, while ignoring non-number characters, which allows the text file to have clear labels for each variable. The assigned variables are then printed to the terminal for the user to see.

- **Avg()**

This function simply finds the mean between two inputs.

- **Export_data()**

This function will either initialise or append to a text file as the outputs of the simulation. In the code, it is called in the `main( )` function to initialise a new text file, writing headers and initial values to the file, then it is called inside the `verlet( )` function to append the parameters of the system at certain timesteps. It contains checks to see if the text file has been opened properly, stopping the code from running if an error occurs.

- **Progress_bar()**

A progress bar has been implemented to the terminal screen to give the user a visual representation of how much progress the code has made through the Verlet integration. It works by rewriting over the previous bar to update the shown progress.

2.6 Plotting

All the plots shown in this report with created in a Jupyter notebook “SPH_Plots.ipynb”, using Matplotlib, an example has been left in the submitted file and plots in this report can be recreated by running the C++ simulation with the specified parameters and running the Jupyter notebook.

Animations of the simulations are also available by running, “SPH_animations.py”. Animations of the systems modelled in this report are also available in the file named “Animations”. The functions used for plots and animations can be found in the “Plotting_Code.py” file.

2.7 Main Code

The `main()` function in the C++ code prints text to the terminal, intended for the user to know if the program is running or not. Inputs are then assigned to variables using the `read_inputs()` function before making and initialising a shock tube system. The initial values of the system are appended to an output file, and finally the `verlet()` function iterates the system through time. Once the code finishes running, the output text file is ready to be read by the Python plotting code.

3 Code Validation

Sod’s shock tube has analytical solutions to each of the five unique sections of the problem. The SPH code was validated by ensuring that the simulated results match the analytical ones.

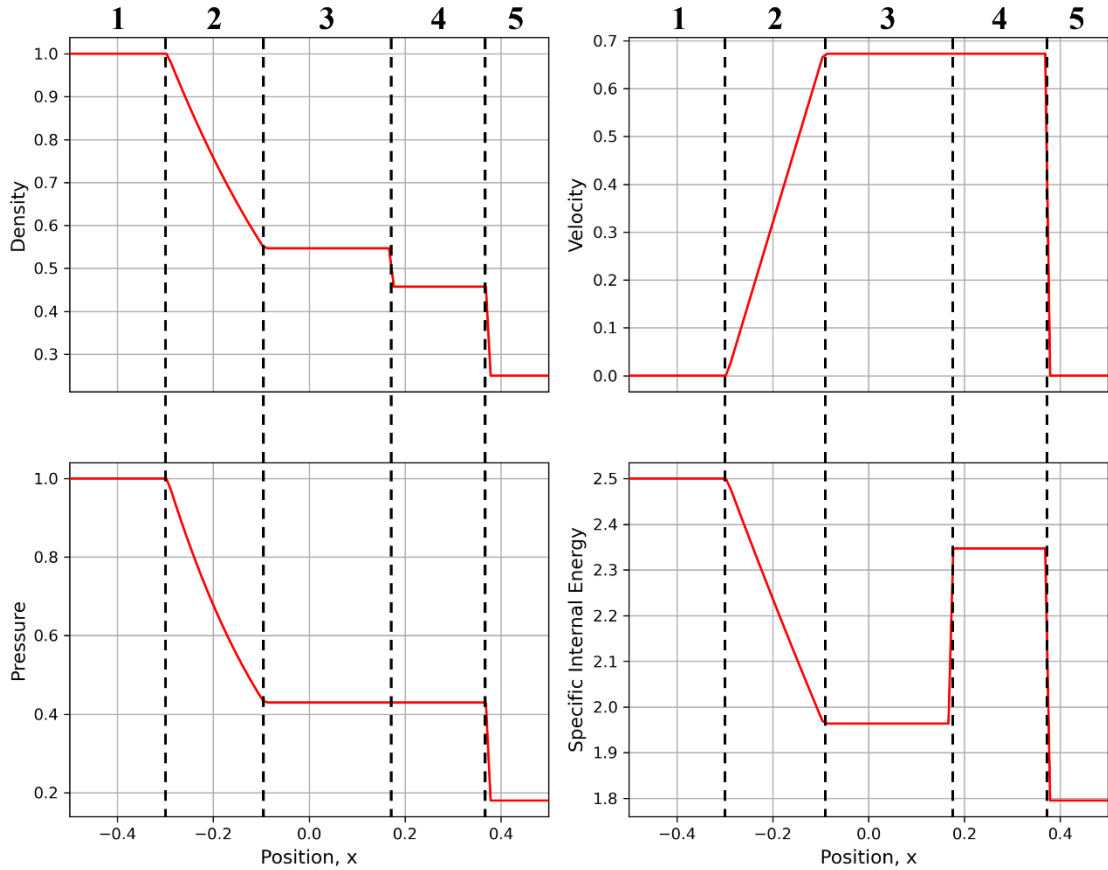


Figure 2: The analytical solutions to Sod’s shock tube problem, with dashed lines showing each unique section of the tube. Section 2 shows the rarefaction fan, between sections 3 and 4 is the contact discontinuity and the point between section 4 and 5 shows the shock position.

The values for density, velocity, pressure and specific internal energy for each of the sections in figure 2 were found using the following equations [3],

$$1: \quad \rho_1 = \rho_H, \quad P_1 = P_H, \quad u_1 = \frac{P_1}{(\gamma-1)\rho_1}, \quad v_1 = 0, \quad (10a)$$

$$2: \quad \rho_2 = \rho_1 \left(1 - \frac{\gamma-1}{2} \frac{v_2}{c_1}\right)^{1/\beta\gamma}, \quad P_2 = P_1 \left(1 - \frac{\gamma-1}{2} \frac{v_2}{c_1}\right)^{1/\beta}, \quad u_2 = \frac{P_2}{(\gamma-1)\rho_1}, \quad v_2 = \frac{2}{\gamma+1} \left(c_1 + \frac{x-x_{mid}}{t}\right), \quad (10b)$$

$$3: \quad \rho_3 = \rho_1 \left(\frac{P_3}{P_1} \right)^{1/\gamma}, \quad P_3 = P'_3, \quad u_3 = \frac{P_3}{(\gamma-1)\rho_3}, \quad v_3 = \left(\frac{2c_1}{\gamma-1} \right) \left(\left(\frac{P'_3}{P_1} \right)^\beta - 1 \right), \quad (10c)$$

$$4: \quad \rho_4 = \rho_5 \left(\frac{P_4 + \Gamma P_5}{P_5 + \Gamma P_4} \right), \quad P_4 = P_3, \quad u_4 = \frac{P_4}{(\gamma-1)\rho_4}, \quad v_4 = (P'_3 - P_5) \sqrt{\frac{2}{(\gamma+1)\rho_5(P'_3 + \Gamma P_5)}}, \quad (10d)$$

$$5: \quad \rho_5 = \rho_L, \quad P_5 = P_L, \quad u_5 = \frac{P_5}{(\gamma-1)\rho_5}, \quad v_5 = 0, \quad (10e)$$

$$\text{with } \Gamma = \frac{\gamma-1}{\gamma+1} \text{ and } \beta = \frac{\gamma-1}{2\gamma}, \quad (10f)$$

where ρ_n, P_n, u_n and v_n denotes the density, pressure, specific internal energy and velocity in the n th section of the tube, respectively. γ represents the adiabatic index, ρ_H, P_H represent the higher initial density and pressure in the tube, and ρ_L, P_L represent the lower initial density and pressure in the tube. x denotes the position of a particle in the tube, x_{mid} denotes the initial discontinuity point in the tube and t represents the time. The velocities, v_3 and v_4 need to be iterated over until they are equal, the value of P'_3 that allows this the pressure in the third section of the tube.

The function “analytical_shocktube()” found in the “Plotting_Code.py” file, encodes the full set of eq(10) for plotting the analytical solution. It calls the function “find_p_iterative()” which uses the Newton-Raphson method to find the correct pressure and velocities. It also calls the function “sound_speed()” which encodes eq(9) in Python.

Using the parameters given in table 1, the simulated shock tube initially looks like a step function for density, pressure and specific internal energy, with all velocities starting at zero, as seen in figure 3. Unless stated otherwise, all simulations used these starting conditions, so all plots of the initial values would be the same.

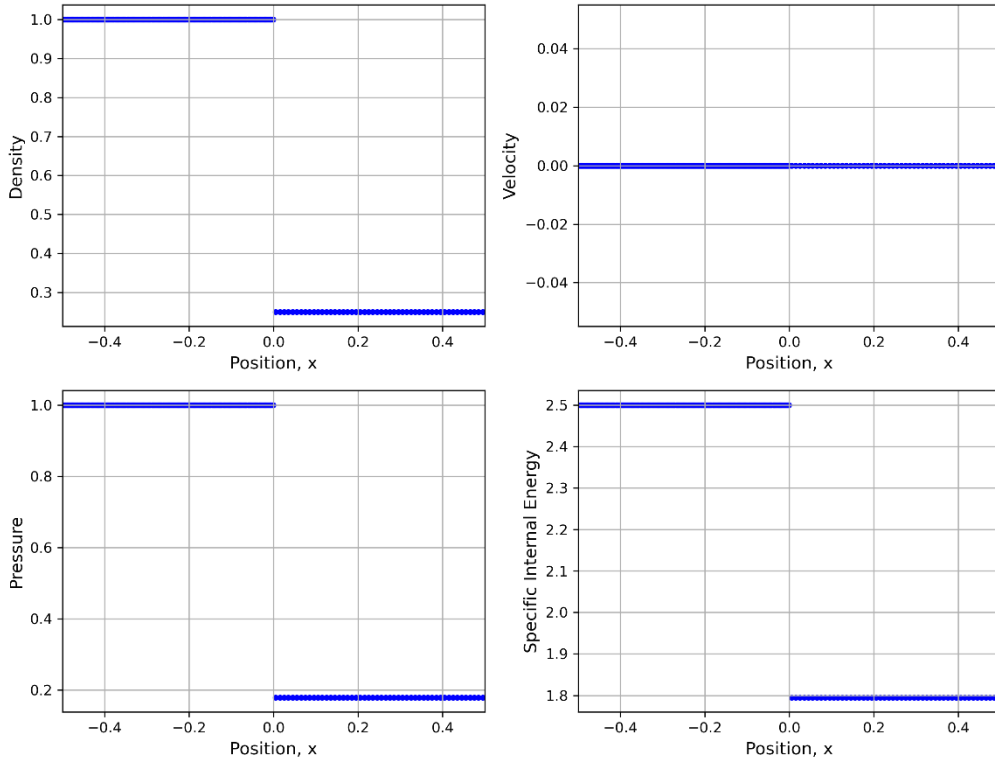
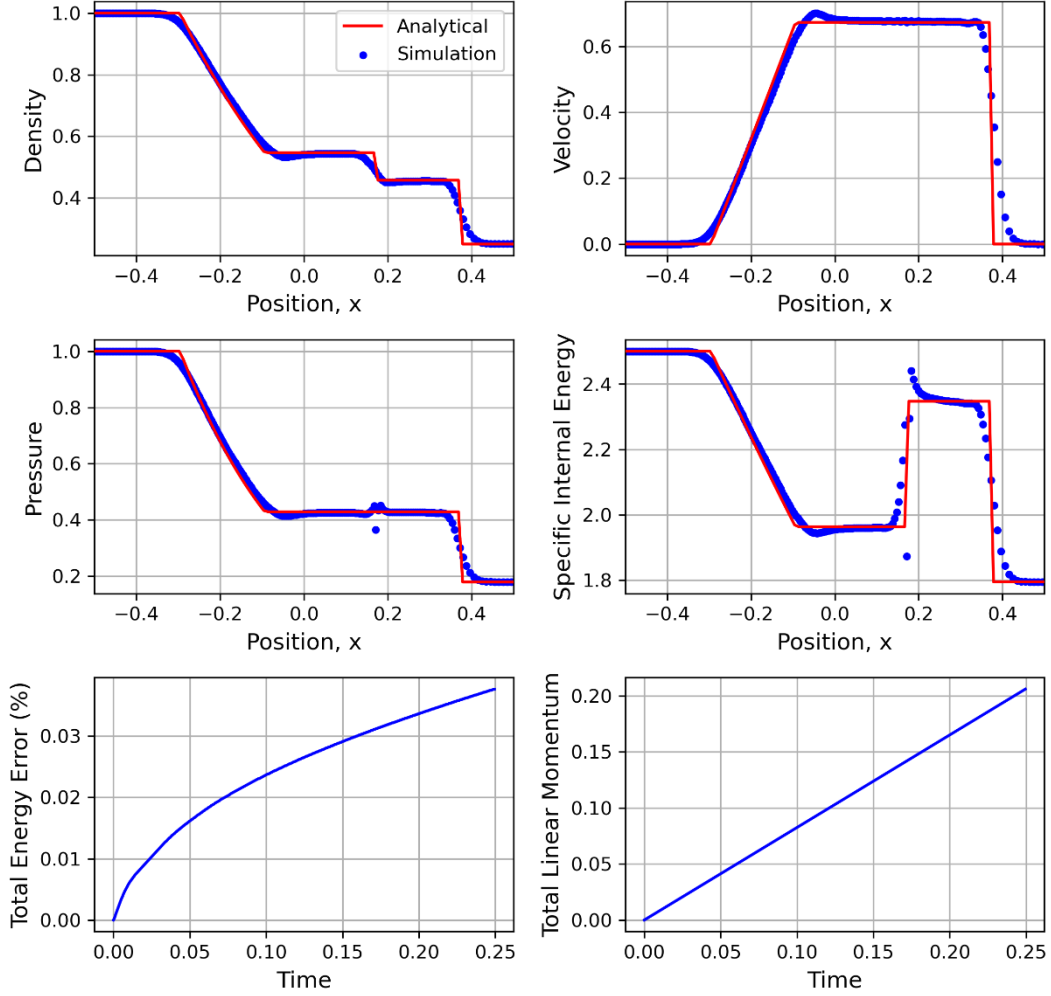


Figure 3: The initial values for the shock tube problem, using the parameters given in table 1.

Running the model for $t=0.25$, figure 4 shows the state of the shock tube at the end of the simulation. There is very good consistency between the analytical and simulated solutions, verifying that the code is working correctly. The percentage error in the energy increases through time, with the maximum error being 0.038% . The total energy during a Verlet integrator is expected to fluctuate over time but oscillate back to the correct value. Due to the time the simulation runs for being short, it

is possible that there is not enough time for the energy to have returned to its starting value. This, in combination with the error being very small (less than 1%), leads to high confidence in the presented



simulation.

Figure 4: The values of the shock tube at $t=0.25$, using the parameters given in table 1. The analytical solutions at the same time are overlayed on top of the simulated particles. Total percentage energy error and total linear momentum throughout the simulation are also plotted on the bottom two graphs.

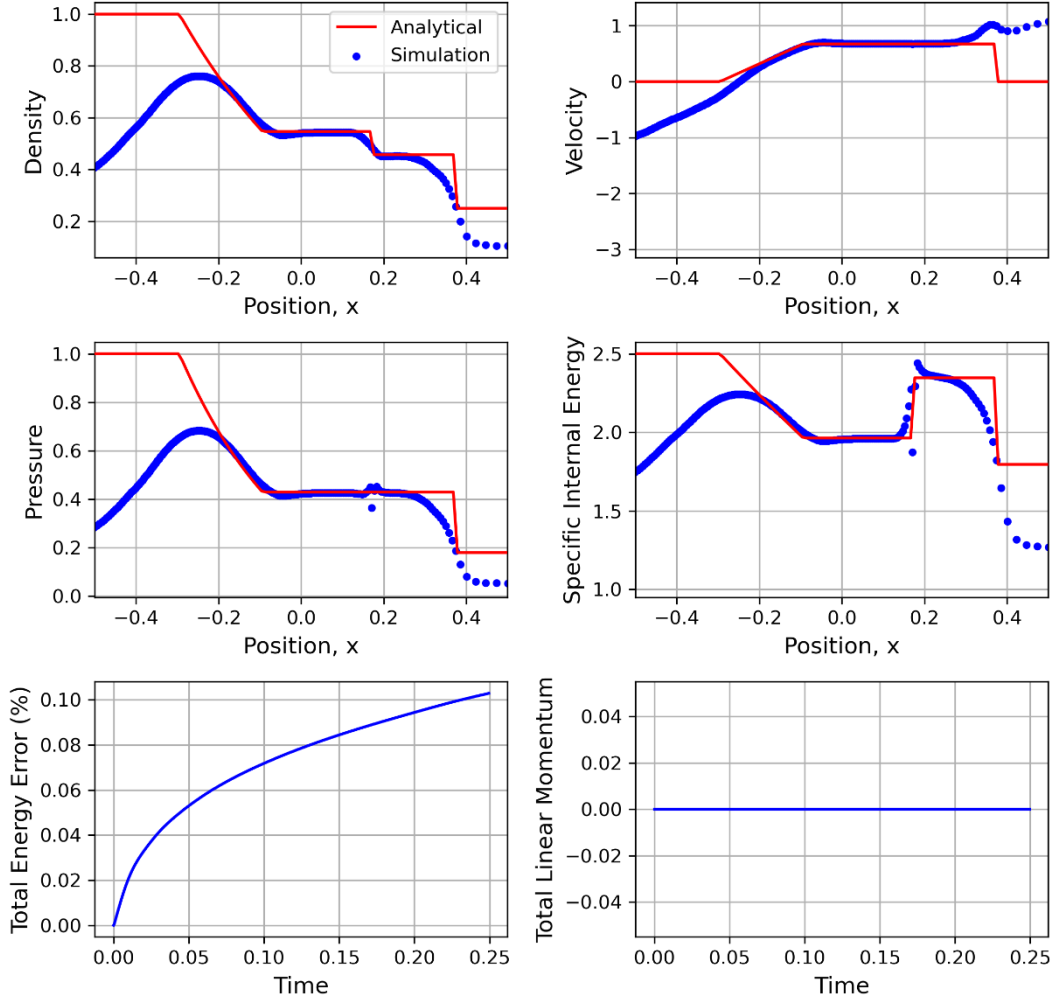
The total linear momentum in the shock tube is seen to grow linearly with time through the simulation. This is due to the interaction between the particles modelling the shock tube, and any boundary particles present. This arises from the interaction between these particles when calculating values such as density and pressure which change for model particles, but boundary particles are set to static parameters to represent the wall of the tube, resulting in some momentum being added to the system each timestep without being cancelled out by another particle. This extra value is not accounted for in the initial parameters, which is why the momentum starts at zero, but this change does not affect the model's performance and total linear momentum is still considered to be conserved throughout the Verlet integration for models that run over a low total amount of time.

An animation of the simulation in this case is provided in the handed in zip folder, it is named "Final_Animation.gif" and is found in the folder labelled "Animations".

3.1 Case of no boundary particles

Running the SPH code again, this time without any boundary particles present. The shock tube runs as expected. With the shock, contact discontinuity and rarefaction fan all being present in the plotted solutions shown in figure 5. However, due to no particles being present outside of the tube, the

experienced density and pressure for particles on the edges of the tube are lower leading to a wave of particles moving away from the tube and reducing their densities, pressures and specific internal



energies, while increasing their speeds away from the tube.

Figure 5: The shock tube simulation at $t=0.25$, when there are no boundary particles present, all other parameters are as given in table 1. The analytical solutions at the same time are overlayed on top of the simulated particles. Total percentage energy error and total linear momentum throughout the simulation are also plotted on the bottom two graphs.

Despite this, the Verlet integrator has conserved energy to below a 1% error and conserves total linear momentum, with no linear rate of change due to no boundary particles being present. This provides more evidence that the Velocity-Verlet integrator is working correctly, ensuring that energy and momentum are being conserved.

An animation of the simulation in this case is provided in the handed in zip folder, it is named “No_Boundary_Animation.gif” and is found in the folder labelled “Animations”.

3.2 Case of no artificial viscosity

Keeping all the parameters the same as in table 1, except for turning artificial viscosity off produces the plots seen in figure 6. The simulated shock tube largely remains in agreement with the analytical solutions, however large oscillations are observed at the shock location, which is the point of discontinuity between sections 4 and 5, especially for the velocities of the particles. This type of behaviour for the SPH code is expected for a shock tube and a similar result for the velocity can be found in the paper *Astrophysical Smoothed Particle Hydrodynamics* [1], giving legitimacy to the results created by this SPH code.

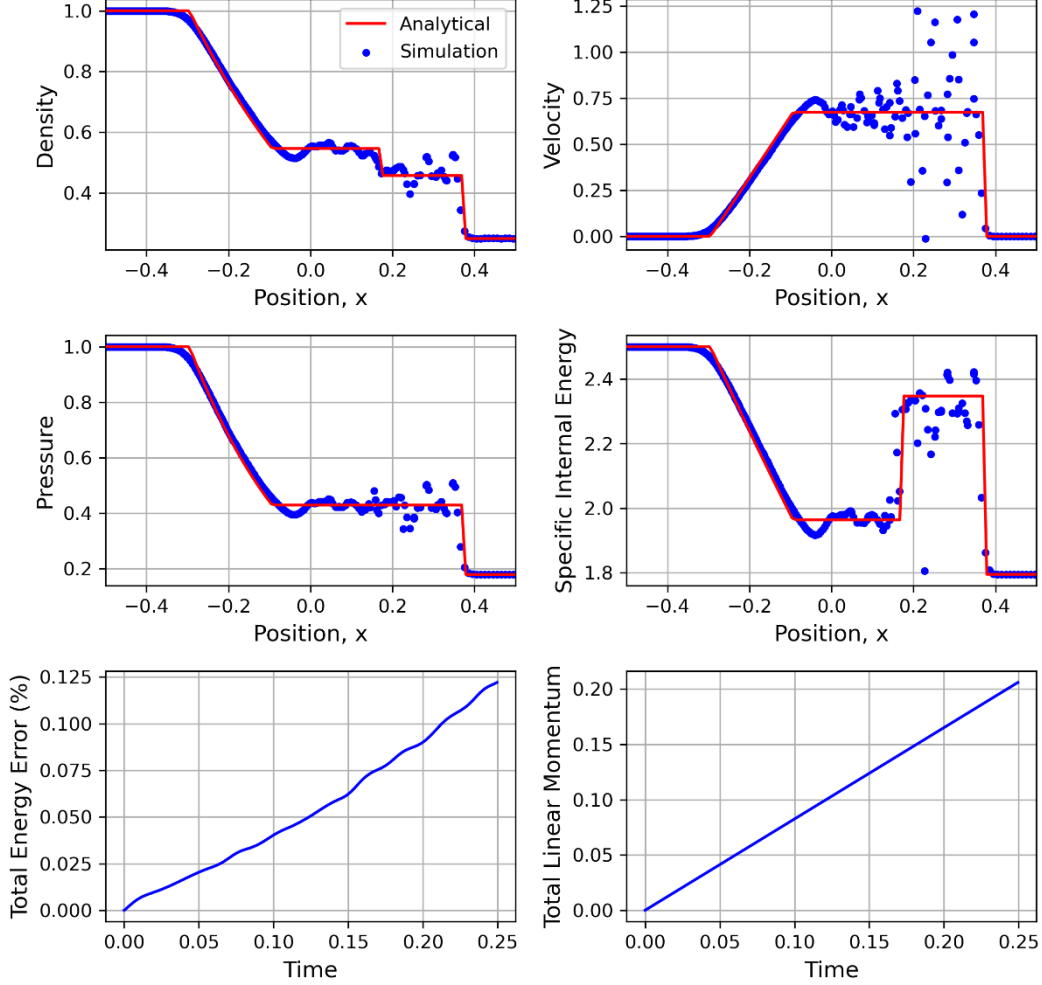


Figure 6: The shock tube simulation at $t=0.25$, with no artificial viscosity, all other parameters are as given in table 1. The analytical solutions at the same time are overlayed on top of the simulated particles. Total percentage energy error and total linear momentum throughout the simulation are also plotted on the bottom two graphs.

An animation of the simulation in this case is provided in the handed in zip folder, it is named “No_Viscosity_Animation.gif” and is found in the folder labelled “Animations”.

4 Code Convergence

The timestep used in this report is 0.0005 , given in table 1. This was found to work well for the SPH code through the use of code convergence, where a larger initial timestep was used and reduced until the Verlet integrator worked as expected. Given the total time for the simulation is $t=0.25$, initially a timestep of 0.01 was used.

Figure 7a shows that the shock tube system blows up during the simulation, leaving no particles in the expected range at the end point. The point where the simulation failed can be seen using the total energy error graph; after a point, the energy starts to rapidly grow once the system explodes with high velocities. The timestep was too large to effectively resolve the behaviour at the shock point. Halving the timestep to 0.005 , as seen in figure 7b, accurately produces results that agree with the analytical solutions, however the energy error grew quickly, so a smaller timestep was used to reduce this.

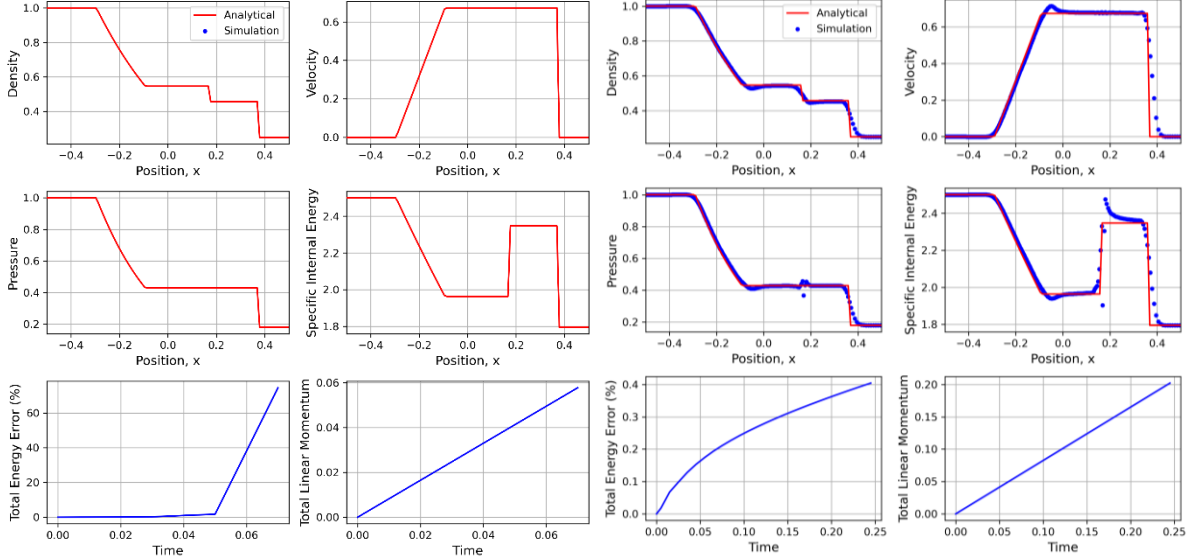


Figure 7: Both plots show the values of the particles in the shock tube at time, $t=0.25$. (a) The 6 plots on the left show the values of the particles when a timestep of 0.01 is used. (b) The 6 plots on the right show the values of the particles when a timestep of 0.005 is used, showing much greater accuracy than the higher timestep.

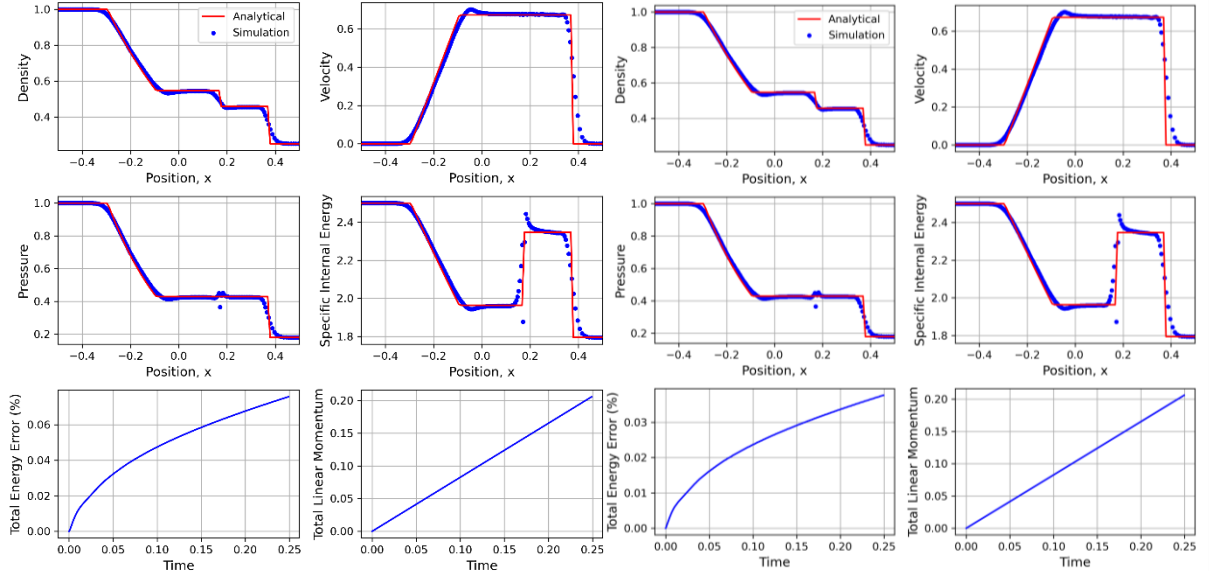


Figure 8: Both plots show the values of the particles in the shock tube at time, $t=0.25$. (a) The 6 plots on the left show the values of the particles when a timestep of 0.001 is used. (b) The 6 plots on the right show the values of the particles when a timestep of 0.0005 is used.

Figure 8a shows the results of using a timestep of 0.001 . High agreement between analytical and simulated solutions is kept, while significantly reducing the percentage error on the energy. It should be noted that the timestep has no effect on the error of the total linear momentum in the system. For a constant smoothing length, this timestep would be suitable for running this SPH code, but halving it once more, as seen in figure 8b, does not significantly impact computing time, as the program runs quickly anyway. And it allows for accurate use of variable smoothing lengths, so for simplicity in the report, all presented results are given using the timestep of 0.0005 .

5 Extension – Variable Smoothing Lengths

The extension for this assessment was chosen to be the implementation of variable smoothing lengths to the SPH code. In 1D, variable smoothing lengths can be calculated using the following equation [1],

$$h_i = \eta \frac{m_i}{\rho_i}, \quad (11)$$

where h is the smoothing length, m is the mass, ρ is the density, η is a constant between 1.2 and 1.5, and the subscript i denotes a specific particle.

In the code, variable smoothing lengths are only active if the toggle is set to true. If it is true, a function called “variable_h()” is called, which encodes eq(11). This happens in two places in the code; in the initialise() class method, and in the find_params() class method, both times the current density is used to find the value of the smoothing length for each particle. When calculating the artificial viscosity, acceleration and power, the average smoothing length is used in these functions to help conserve energy and momentum, however this method of implementation does not work perfectly, and energy is not exactly conserved. However, this does not have a significant impact on the code due to the small lifetime of the simulation.

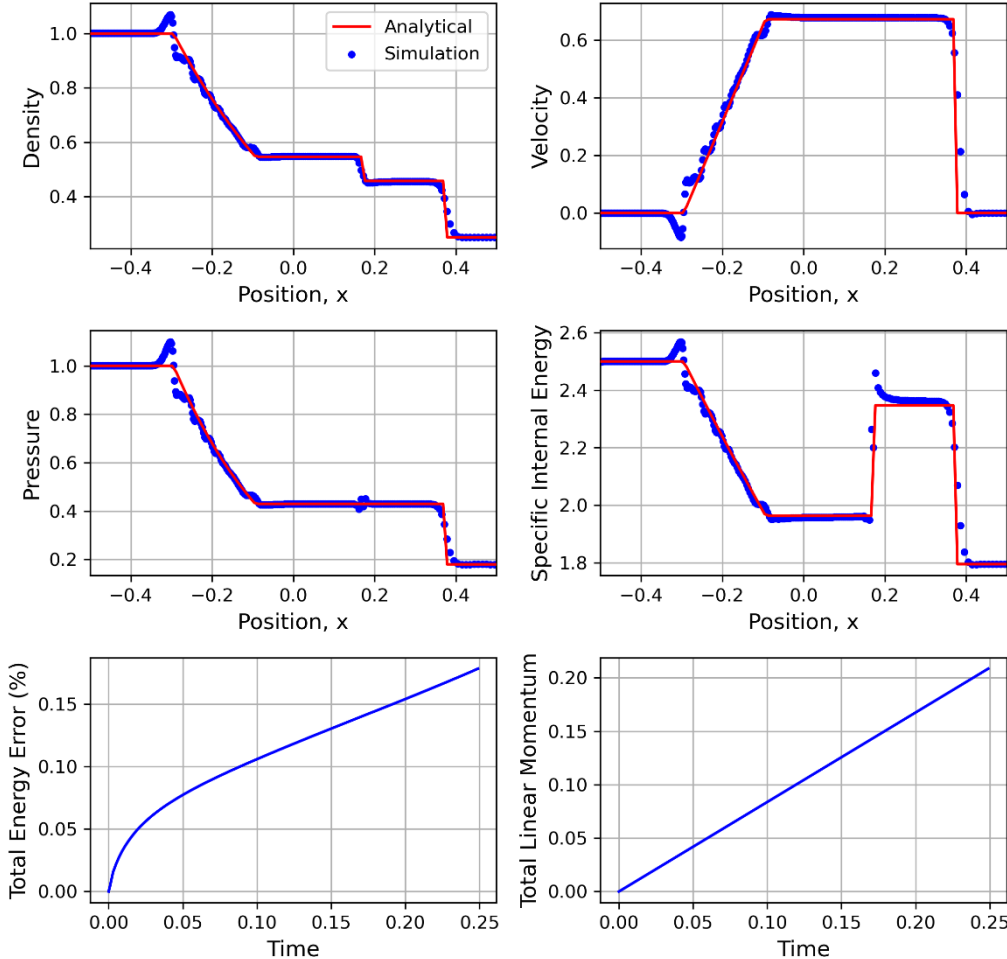


Figure 9: Plots of the values for each particle in the shock tube at $t=0.25$, using variable smoothing lengths and a timestep of 0.001 . The analytical solutions are overlayed on top of the SPH simulation solutions.

Variable smoothing lengths lead to a higher error in the total energy, this can be seen by comparing the total energy error between figures 8a and 9, which use the same timestep. For this

reason, the timesteps were lowered to 0.0005 , as seen in figure 10, to keep a small energy error in all cases.

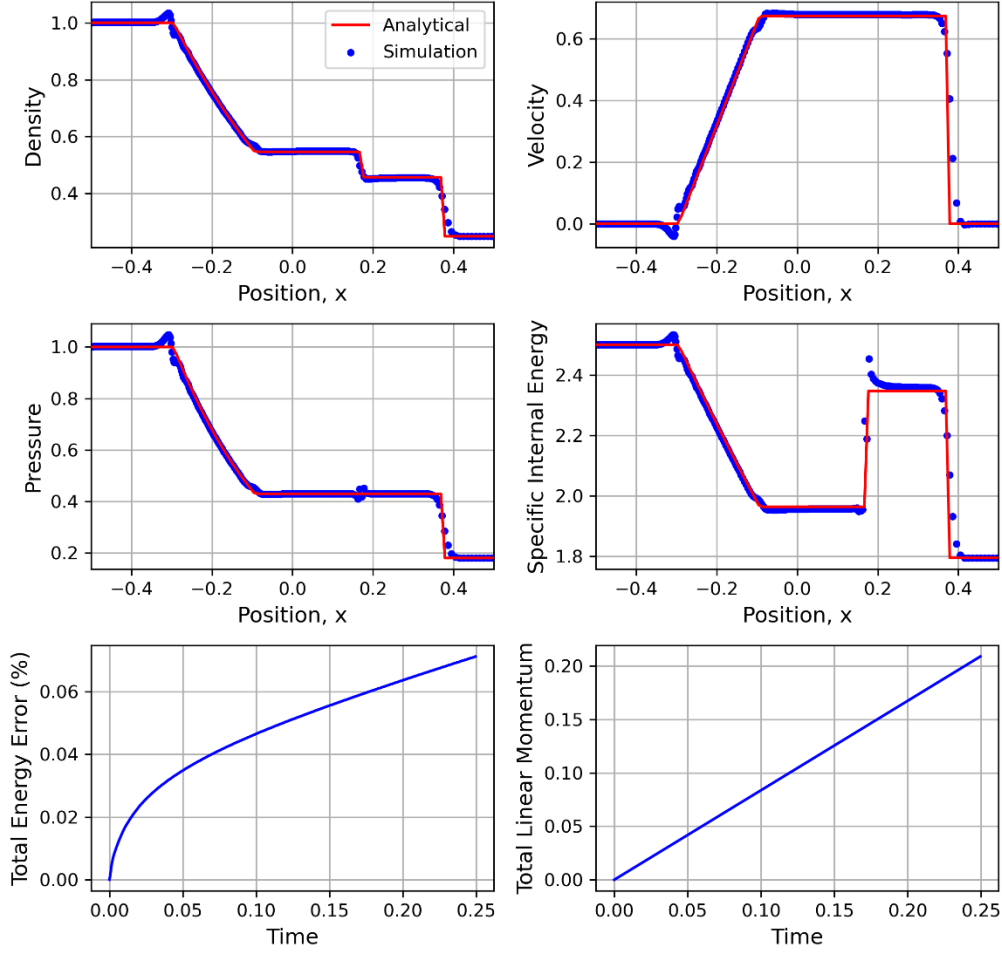


Figure 10: Plots of the values for each particle in the shock tube at $t=0.25$, using variable smoothing lengths and a timestep of 0.0005 . The analytical solutions are overlayed on top of the SPH simulation solutions.

The stability of the features in the plots in figures 9 and 10 show how a smaller timestep helps the code remain stable during its runtime. Plots in figure 9 show the rarefaction fan as having small oscillations within itself, which can lead to instability and later errors in the code. A smaller timestep, seen in figure 10, helps smooth out these extra oscillations, providing a more realistic representation of the shock tube.

In general, the differences between constant and variable smoothing lengths are hard to see, especially one a smaller scale such as with a shock tube, however there are some key differences that can be seen. Such as, the lack of floating particles and a lip occurring at the top of the rarefaction fan, in the variable smoothing length cases.

An animation of the simulation in this case is provided in the handed in zip folder, it is named “Variable_h_Animation.gif” and is found in the folder labelled “Animations”.

6 References

- [1] Price DJ, Wurster J, Tricco TS, Nixon C, Toupin S, Pettitt A, Chan C, Mentiplay D, Laibe G, Glover S, Dobbs C. Phantom: A smoothed particle hydrodynamics and magnetohydrodynamics code for astrophysics. Publications of the Astronomical Society of Australia. 2018 Jan;35:e031. Available from: <https://arxiv.org/pdf/1702.03930.pdf>
- [2] Rosswog S. Astrophysical smooth particle hydrodynamics. New Astronomy Reviews. 2009 Apr 1;53(4-6):78-104. Available from: <https://arxiv.org/abs/0903.5075>
- [3] Toro EF. Riemann solvers and numerical methods for fluid dynamics: a practical introduction. Springer Science & Business Media; 2013 Apr 17. Available from: <https://link.springer.com/book/10.1007/b79761>